School of Future Tech
Case Study Report

Case Study 10: Banking Interest Calculator

Kunwar Bhosle
Sam Altman
150096725181

Index

1. Introduction to the Case Study.
2. Problem Statement
3. Methods & Algorithms Technology Applied in the Problem Statement /
Case Study.
4. Problem Statement / Case Study Implementation Details and Snapshots.
5. Case Study Results and Conclusion.

1.Introduction to the Case Study :

In modern banking systems, calculating interest accurately is an essential financial operation. Banks use compound interest to calculate the interest on savings accounts, fixed deposits, and loans. Compound interest means that interest is calculated not only on the initial principal but also on the accumulated interest from previous periods.

Manually calculating compound interest for different values of principal, rate, and time can be time-consuming and prone to human errors. Therefore, computerized solutions are widely used to automate these calculations.

This project presents a Bank Compound Interest Calculator developed using the C++ programming language. The program allows the user to input the principal amount, rate of interest, and time period, and then calculates both the compound interest and the final amount automatically.

The project demonstrates how basic programming concepts such as functions, input/output operations, mathematical calculations, and variables can be used to solve real-world financial problems efficiently.

2. Problem Statement / Case Background (Abstract) :

Banks and financial institutions frequently need to calculate compound interest for customers. When done manually, this process can be slow and susceptible to calculation errors.

The problem addressed in this case study is:

To develop a simple C++ program that calculates compound interest and the total amount based on user inputs for principal, interest rate, and time period.

The program should:

- Accept **principal amount** from the user
- Accept **rate of interest**
- Accept **time period in years**
- Calculate **compound interest using the compound interest formula**
- Display both **compound interest and final amount**

This program helps automate interest calculations and demonstrates how programming can be applied in banking operations.

3. Methods & Algorithms Technology Applied in the
Problem Statement / Case Study :

Programming Language Used

The project is implemented using **C++**, a powerful object-oriented programming language commonly used for system programming and application development.

Libraries Used

1.iostream

Used for input and output operations.

Allows the program to display messages and receive user input.

1.cmath

Used for mathematical functions.

The `pow()` function is used to calculate powers.

Algorithm Used

Step-by-Step Algorithm

1. Start the program.
2. Declare variables:

    `principal`

    `rate`

    `time`

    `interest`

    `amount`

1. Ask the user to enter:

    Principal amount

    Rate of interest

    Time in years

1. Call a function named **Interest()** to calculate compound interest.
2. Inside the function:

    Use the compound interest formula.

1. Return the calculated interest value.
2. Calculate the final amount:Amount = Principal + Interest
3. Display the compound interest.
4. Display the total amount.
5. End the program.

Compound Interest Formula Used

Compound Interest is calculated using:

Interest = P(1 + r/100)^t − P

Where:

- **P** = Principal amount
- **r** = Rate of interest
- **t** = Time in years

4. Problem Statement / Case Study Implementation Details and Snapshots :

Main_Project_C++ > amount.cpp > ...

```cpp
#include <iostream>
#include <cmath>
using namespace std;

double Interest(double principal, double rate, double time)
{
    double interest,amount;
    interest= principal*(pow((1 + rate/100),time))-principal;
    return interest;
}

int main()
{
    double principal, rate, time, interest, amount;

    cout<<"Enter the principal amount: ";
    cin>>principal;

    cout<<"Enter the Rate: ";
    cin>>rate;

    cout<<"Enter the time: ";
    cin>>time;

    interest= Interest(principal, rate, time);
    amount=interest+ principal;

    cout<<"The Compount Interest for "<<time<<" years is: "<<interest<<endl;
    cout<<"The Amount for "<<interest<<" interest is: "<<amount<<endl;

}
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} C++ Go Live Mac

double Interest() Function:

This function:

Takes **principal, rate, and time** as parameters

Uses the **pow() function** to calculate exponentiation

Returns the calculated **compound interest**

int main() Function:

The `main()` function handles user input and displays results.

The program then calls the function:

Finally, the program prints the results:

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

cd "/Users/kunwarbhosle/Desktop/C++/Main_Project_C++/" && g++ amount.cpp -o amount && "/Users/kunwarbhosle/Desktop/C+
/Main_Project_C++/"amount
kunwarbhosle@Kunwars-MacBook-Air C++ % cd "/Users/kunwarbhosle/Desktop/C++/Main_Project_C++/" && g++ amount.cpp -o am
unt && "/Users/kunwarbhosle/Desktop/C++/Main_Project_C++/"amount
Enter the principal amount: 5000
Enter the Rate: 8
Enter the time: 3
The Compount Interest for 3 years is: 1298.56
The Amount for 1298.56 interest is: 6298.56
kunwarbhosle@Kunwars-MacBook-Air Main_Project_C++ %
```

5.Conclusion :

The Bank Compound Interest Calculator successfully performs the following tasks:

- Accepts user input for principal, rate, and time
- Calculates compound interest accurately
- Calculates total amount after interest
- Displays results clearly to the user

The program demonstrates that basic programming techniques can be effectively applied to solve financial calculation problems.

Conclusion

The Compound Interest Calculator is a simple yet practical application of C++ programming in the banking domain. It automates interest calculations and eliminates the possibility of manual calculation errors.

Through this project, the following programming concepts were applied:

- Functions
- Mathematical operations
- Input and output handling
- Use of libraries
- Program structure and logic

This case study shows how programming can help simplify real-world financial computations and improve efficiency in banking operations.